

What are some other modes of access that users might want to apply to code or data, in addition to the common read, write, and execute permission?

Apart from the standard rwx permissions, implementing more specialized access modes such as *append*, *delete*, *audit* and *granular actions* would be more helpful and provide robust mechanism to protect system and its assets. If implemented and enforced properly, the additional modes could prevent many unwanted action from a user and/program in thus preventing unexpected behavior from the system as a whole.

1. **Append** allows a user to add new data to file while not allowing modify existing content of it. Even though it may not be a feature/measure that is constantly used by user or user-end programs, enforcing this feature helps avoid accidental activities leading to modify a file as a whole where it was supposed to just attach new data to it.
2. **Delete** prevents a user from deleting a file specifically not authorized to do so. Many times we often encounter accessing files that doesn't directly relate to us and we end of deleting it. Now, although OS related files are protected by OS local accounts, deletion is generally accepted for overall files (system and/or user) as long as the file is closed. Proper enforcement of the deletion mode (either by OS or by user programs) would prevent any active user from deleting it and thus, preventing any unexpected behavior from the system (OS, programs) relying on it.
3. **Granular actions** on the other hand is an absolutely brilliant way of defining actions and behavior of an object, and authorizing those actions to a user. For example, there could be a *truncate* action for a directory where the action if executed, would truncate/empty the directory without removing it of course. It is to keep in mind that truncate in this scenario has nothing to do with delete or rename. Meaning, it is an independent action of an object that has its own authorization level implemented to provide more detailed action.

Give an example of the use of physical separation for security in a computing environment.

A classic example of physical separation can be seen in big institutions such as big private organization, secure government or financial institutions where they host their own work and management system that is secured and isolated from public networks. For example, Area 51 (a classified military location) has its systems placed in a secure room with no external connectivity, required authorized personnel to access it in person. Also, access and works are compartmentalized to prevent escalations. This simply means that even if the organization's publicly available system is compromised, the critical classified and isolated data remains intact from external threats.

Give an example of the use of temporal separation for security in a computing environment.

Temporal separation occurs when multiple users or processes use the same resource at different times to avoid interference or leakage. In simple terms, it is a logical separation of code execution and data flow among programs. A practical example is found in shared high-performance computing clusters used for research. Instead of allowing multiple sensitive computations to run simultaneously, the scheduler allocates exclusive time slots to different projects—especially those handling confidential or export-controlled data. By ensuring that sensitive processes are executed in isolated time windows, organizations minimize risks of data leakage through shared memory, timing channels, or resource contention.

Do you think attempting to break into (that is, obtain access to or use of) a computing system without authorization should be illegal? Why or why not?

Yes, unauthorized access should absolutely be illegal because it violates trust, privacy, and system stability, even when no direct harm is done. Computing systems often hold sensitive personal data, critical operational information, or proprietary business logic. When someone accesses these systems without permission, they bypass not only technical barriers but also legal and ethical boundaries designed to protect individuals and organizations. Even harmless intrusions can unintentionally expose vulnerabilities or disrupt services. However, laws should also differentiate between malicious intent and legitimate security research, ensuring that ethical hacking under proper disclosure frameworks is encouraged rather than criminalized.

Can a database contain two identical records without a negative effect on the integrity of the database? Why or why not?

Yes, a database can contain two identical records, but doing so can threaten its integrity depending on the system's design and purpose. If there's no unique identifier or primary key to distinguish one record from another, duplicate entries can lead to inconsistencies in reporting, errors in processing, and ambiguity in decision-making. For example, if two identical patient records exist in a healthcare system, it could cause confusion about treatment history or billing. On the other hand, some systems deliberately allow duplicates under controlled conditions such as logging repeated identical transactions because the context preserves integrity. Nevertheless, integrity depends less on the duplication itself and more on the database's ability to maintain clear and reliable relationships between data.